DailyQuest — Technical Project Summary

Comprehensive technical documentation and architectural analysis of the DailyQuest full-stack application as of October 2026.

---

1. PROJECT OVERVIEW

Description

DailyQuest is a gamified productivity and routine tracking application that models personal habits, scheduled tasks, and focus sessions as an RPG-style quest system. Users create recurring or one-time routines, earn Experience Points (XP) proportionally based on task completion, advance through numerical adventurer levels, maintain consecutive daily streaks (governed by an 80% daily completion threshold), and unlock achievement badges. The app features a dedicated Focus Room with a multi-phase Pomodoro timer, live ambient audio chimes, historical completion analytics with GitHub-style heatmaps, a structured activity audit table, end-of-day report grading (S/A/B/C/D), AI-powered productivity summaries and suggestions, and bi-directional offline synchronization.

Tech Stack

- **Frontend Framework**: Next.js 14.2.15 using the App Router (`src/app`), React 18.3.1, React DOM 18.3.1. (A secondary Vite 6.3.5 configuration exists in `vite.config.js` for standalone builds).
- **Backend Framework**: Express 4.21.2 running on Node.js with native ES Modules (`"type": "module"`).
- **Database & ODM**: MongoDB via Mongoose 8.9.5. In development, the backend automatically provisions a local persistent database via `mongodb-memory-server` 10.1.4 (persisted to `server/.mongo-data` using the WiredTiger storage engine) when no external connection string is provided.
- **Authentication**: Stateless JSON Web Tokens (`jsonwebtoken` 9.0.2) signed with HS256 and salted password hashing via `bcrypt` 5.1.1 (10 salt rounds). Transmitted via the `Authorization: Bearer <token>` HTTP header. Also features a local-only Guest Exploration mode that operates without server credentials.
- **Hosting / Dev Setup**:

- Web client: `http://localhost:3000` via Next.js development server (`npm run dev`).

- API server: `http://localhost:4000/api/v1` via Node.js file-watching (`npm --prefix server run dev`).

- Concurrent launcher: `npm run dev:all` using `concurrently` 10.0.5.

---

2. FOLDER STRUCTURE

```
APP-junior/
├── .freebuff/              # IDE and workspace environment metadata
├── .next/                  # Next.js App Router build cache and compiled server/client bundles
├── dist/                   # Static build output generated by the Vite build pipeline
├── node_modules/           # Root workspace npm dependencies
├── server/                 # Express.js REST API backend service
│   ├── node_modules/       # Backend-specific npm dependencies
│   ├── src/
│   │   ├── config/         # Environment variables, database connection, and constants
│   │   ├── controllers/    # Express route controller handlers
│   │   ├── middleware/     # Auth, validation, error handling, and 404 middlewares
│   │   ├── models/         # Mongoose database models (User, State, AiCache)
│   │   ├── routes/         # Express endpoint route definitions grouped by resource
│   │   ├── services/       # Core business logic and external service integrations
│   │   ├── utils/          # Mathematical algorithms (XP, streaks, schedule, grades)
│   │   ├── app.js          # Express application initialization and middleware setup
│   │   └── index.js        # Server entrypoint and HTTP listener
│   ├── test/               # Backend automated test suite (node:test)
│   └── package.json        # Backend package definition and scripts
├── src/                    # Frontend application source code
│   ├── api/                # Unified frontend API client and error wrappers
│   ├── app/                # Next.js App Router routes, layouts, and loading states
│   │   ├── achievements/   # /achievements route (Trophies & Badges)
│   │   ├── actions/        # Next.js Server Actions for data prefetching
│   │   ├── analytics/      # /analytics route (Heatmap, Trends, Audit Table)
│   │   ├── focus/          # /focus route (Pomodoro Focus Chamber)
│   │   ├── login/          # /login route (Split-screen authentication view)
│   │   ├── quests/         # /quests route (Main dashboard & task management)
│   │   ├── settings/       # /settings route (Preferences, theme, backup)
│   │   ├── globals.css     # Global styles and Tailwind CSS v4 custom theme definitions
│   │   ├── layout.jsx      # Root HTML layout, font setup, and top-level providers
│   │   └── page.jsx        # Root route redirector (/ to /quests or /login)
│   ├── components/         # React component tree
│   │   ├── achievements/   # Achievement view components
│   │   ├── charts/         # Recharts trend data visualizations (Daily, Weekly, Monthly)
│   │   ├── layout/         # AppShell, persistent Sidebar, TopBar, and BottomNav
│   │   ├── quests/         # Client-side quest dashboard container
│   │   └── ui/             # Reusable design system library (cards, headers, dialogs)
│   ├── context/            # React Context providers (Auth, Quest, Focus, Modal)
│   ├── hooks/              # Custom application hooks (useAuth, useFocusTimer)
│   ├── utils/              # Client-side utility functions (XP math, streaks, dates, chimes)
│   ├── constants.js        # Shared domain constants (task types, repeat modes, XP caps)
│   └── storage.js          # LocalStorage caching layer and background server synchronization
├── .env.example            # Template for backend and frontend environment variables
├── next.config.mjs         # Next.js App Router configuration
├── package.json            # Frontend and root workspace configuration
├── postcss.config.mjs      # PostCSS configuration for Tailwind CSS v4
├── tailwind.config.js      # Tailwind configuration file
└── vite.config.js          # Vite legacy/fallback configuration
```

---

3. BACKEND DETAILS

API Routes

All endpoints are mounted under `/api/v1` (with `/health` at root). Protected routes require a valid header: `Authorization: Bearer <jwt>`.

1. Public / System

Method	Path	Description	Auth	Request Shape	Response Shape
`GET`	`/health`	Server uptime check	No	None	`{ ok: true, uptime: number }`
`GET`	`/api/v1/`	API index metadata	No	None	`{ name: "DailyQuest API", version: "v1" }`

2. Authentication (`/api/v1/auth`)

Method	Path	Description	Auth	Request Shape	Response Shape
`POST`	`/api/v1/auth/signup`	Register a new user	No	`{ name: string, password: string }`	`{ user: { id, name, createdAt }, token: string }`
`POST`	`/api/v1/auth/login`	Authenticate existing user	No	`{ name: string, password: string }`	`{ user: { id, name, createdAt }, token: string }`
`GET`	`/api/v1/auth/me`	Retrieve active user profile	Yes	None	`{ user: { id, name, createdAt } }`

3. Tasks (`/api/v1/tasks`)

Method	Path	Description	Auth	Request Shape	Response Shape
`GET`	`/api/v1/tasks`	Retrieve all user tasks	Yes	None	`Array<Task>`
`POST`	`/api/v1/tasks`	Create a new task	Yes	`{ name, emoji?, xp?, type?, target?, unit?, repeat?, days?, onceDate?, startDate?, timeOfDay? }`	`Task`
`PUT`	`/api/v1/tasks/:id`	Update an existing task	Yes	Partial task fields	`Task`
`DELETE`	`/api/v1/tasks/:id`	Delete a task by ID	Yes	None	`{ ok: true, deletedId: string }`

4. Progress Tracking (`/api/v1/progress`)

Method	Path	Description	Auth	Request Shape	Response Shape
`POST`	`/api/v1/progress/:id/complete`	Mark task complete/incomplete	Yes	`{ done?: boolean }`	`{ task, totalXp, level }`
`POST`	`/api/v1/progress/:id/toggle`	Toggle task completion state	Yes	`{ done: boolean }`	`{ task, totalXp, level }`
`PUT`	`/api/v1/progress/:id/count`	Update counter task progress	Yes	`{ value: number }`	`{ task, totalXp, level }`
`PUT`	`/api/v1/progress/:id/minutes`	Update timed task duration	Yes	`{ value: number }`	`{ task, totalXp, level }`
`POST`	`/api/v1/progress/:id/timer`	Start/stop task live timer	Yes	`{ action: 'start' \`	'stop' }`	`{ task, totalXp, level }`

5. User Stats (`/api/v1/stats`)

Method	Path	Description	Auth	Request Shape	Response Shape
`GET`	`/api/v1/stats`	Aggregated user summary	Yes	None	`{ totalXp, level, currentStreak, bestStreak, todayCompletion, totalTasks }`

6. Full State Sync (`/api/v1/state`)

Method	Path	Description	Auth	Request Shape	Response Shape
`GET`	`/api/v1/state`	Fetch full client state doc	Yes	None	`{ version, tasks, totalXp, history, focusSessions, focus, badges, settings, lastActiveDate }`
`PUT`	`/api/v1/state`	Replace full state document	Yes	Full state document	Updated state document
`POST`	`/api/v1/state/migrate`	Import initial localStorage	Yes	`{ state: unknown }`	`{ ok: true, migratedAt: Date, state }`

7. Focus Chamber (`/api/v1/focus`)

Method	Path	Description	Auth	Request Shape	Response Shape
`POST`	`/api/v1/focus/start`	Start focus session	Yes	`{ task: { id, name?, emoji? }, mode?, customMin? }`	Active focus session object
`POST`	`/api/v1/focus/stop`	End focus session & grant XP	Yes	`{ completed?: boolean, elapsedMs?: number, pauses?: number }`	`{ session, xpGranted, totalXp }`
`GET`	`/api/v1/focus/sessions`	List logged focus sessions	Yes	Query: `?limit=number&date=YYYY-MM-DD`	`Array<FocusSession>`

8. Reports (`/api/v1/reports`)

Method	Path	Description	Auth	Request Shape	Response Shape
`GET`	`/api/v1/reports/daily`	Compute daily summary	Yes	Query: `?date=YYYY-MM-DD`	`{ date, totalTasks, completedTasks, completionPct, xpEarned, bonusXp, grade, perfect, tasks }`
`GET`	`/api/v1/reports/weekly`	Compute rolling 7-day report	Yes	Query: `?end=YYYY-MM-DD`	`{ start, end, days, totalXp, avgCompletion, bestDay, focusMinutes }`
`POST`	`/api/v1/reports/finish-day`	Commit daily record to history	Yes	None	History entry object

9. AI Copilot (`/api/v1/ai`)

Method	Path	Description	Auth	Request Shape	Response Shape	
`POST`	`/api/v1/ai/insights`	Habit observations & coaching	Yes	None	`{ source: 'claude' \	'rules', insights: Array<{ type, title, message }> }`
`GET`	`/api/v1/ai/weekly-summary`	Executive weekly AI digest	Yes	None	`{ source: 'claude' \	'rules', summary: { title, highlights, recommendations, focusSummary } }`
`POST`	`/api/v1/ai/suggest-tasks`	Recommend complement habits	Yes	None	`{ source: 'claude' \	'rules', suggestions: Array<{ name, emoji, xp, type, target, repeat, rationale }> }`

Database Models & Schemas

1. `User` (`server/src/models/User.js`)
- `name`: `String` (Required, trimmed, max 30 chars) — Display name.
- `nameKey`: `String` (Required, unique, indexed) — Lowercased name for case-insensitive uniqueness.
- `passwordHash`: `String` (Required, hidden via `select: false`) — Bcrypt salt hash.
- `createdAt`, `updatedAt`: `Date` — Managed via Mongoose timestamps.

2. `State` (`server/src/models/State.js`)
- `user`: `ObjectId` (Ref: `User`, required, unique) — 1-to-1 relationship per user.
- `version`: `Number` (Default: 1) — Data migration schema version.
- `tasks`: `Mixed` (Default: `[]`) — Normalized array of active user tasks.
- `totalXp`: `Number` (Default: 0) — Cumulative lifetime XP.
- `history`: `Mixed` (Default: `[]`) — Daily completed history archives.
- `focusSessions`: `Mixed` (Default: `[]`) — Logged focus session records.
- `focus`: `Mixed` (Default: `null`) — Active ongoing focus session state.
- `badges`: `Mixed` (Default: `{}`) — Map of `{ [badgeId]: unlockIsoDate }`.
- `settings`: `Mixed` (Default: `{}`) — User preferences (theme, audio, intervals).
- `lastActiveDate`: `String` (Default: `""`) — YYYY-MM-DD tracking string.
- `migratedAt`: `Date` (Default: `null`) — Timestamp of initial client localStorage sync.
- `createdAt`, `updatedAt`: `Date` — Managed via Mongoose timestamps.

3. `AiCache` (`server/src/models/AiCache.js`)
- `key`: `String` (Required, unique) — Cache slot identifier (`${userId}:${kind}`).
- `user`: `ObjectId` (Required, indexed) — Reference to requesting user.
- `kind`: `String` (Required) — `'insights'`, `'weekly-summary'`, or `'suggest-tasks'`.
- `source`: `String` (Enum: `['claude', 'rules']`, required) — Provenance of the payload.
- `payload`: `Mixed` (Required) — The cached response data.
- `expiresAt`: `Date` (Required) — TTL index (`expireAfterSeconds: 0`) for automatic MongoDB eviction.

Middleware

In Use

1. `cors`: Dynamic origin validation allowing configured origins and non-production localhost ports (`3000`, `3001`, `5173`).
2. `express.json`: Body parser with a `2mb` payload size limit.
3. `requireAuth` (`server/src/middleware/auth.js`): Verifies the JWT Bearer token and assigns `req.user = { id, name }`.
4. `validate` (`server/src/middleware/validate.js`): Zod schema validator for `req.body`, `req.query`, and `req.params`. Strips unknown keys and reassigns sanitized values.
5. `notFound` (`server/src/middleware/notFound.js`): 404 handler that produces `ApiError.notFound()`.

6. `errorHandler` (`server/src/middleware/errorHandler.js`): Centralized error formatter. Converts Zod errors, Mongoose duplicate key errors (code 11000 -> 409 `CONFLICT`), and CastErrors into standardized JSON: `{ message, code, details? }` .

Not Implemented Yet

- **Rate Limiting**: No rate-limiting middleware (e.g. `express-rate-limit`) is installed; `/api/v1/auth` and `/api/v1/ai` endpoints have no throttling.
- **Request Logging**: No structured logger (e.g. `pino-http` or `morgan`) is active; requests are not logged to stdout.
- **Security Headers**: No `helmet` middleware is mounted (only `app.disable('x-powered-by')` is present).

Environment Variables

Variable Name	Required / Optional	Description
`PORT`	Optional	Backend listener port (defaults to `4000`)
`NODE_ENV`	Optional	Environment mode (`'development'` vs `'production'`)
`MONGODB_URI`	Required in Prod	MongoDB connection URI (dev auto-provisions in-memory MongoDB)
`JWT_SECRET`	Required in Prod	Secret key for JWT signing (dev generates an ephemeral secret)
`JWT_EXPIRES_IN`	Optional	JWT duration string (defaults to `'30d'`)
`CLAUDE_API_KEY`	Optional	Anthropic API key for AI endpoints (alias: `ANTHROPIC_API_KEY`)
`CLAUDE_MODEL`	Optional	Model identifier (defaults to `'claude-sonnet-4-5'`)
`AI_CACHE_HOURS`	Optional	TTL cache duration for AI responses (defaults to `4`)
`CORS_ORIGIN`	Optional	Allowed CORS origins (defaults to `http://localhost:3000,http://localhost:5173`)

AI Provider Integration

- **Service**: Anthropic Messages API (`https://api.anthropic.com/v1/messages`).
- **Model**: `claude-sonnet-4-5` (overridable via `CLAUDE_MODEL`).
- **Credential Storage**: Read strictly on the server from `CLAUDE_API_KEY` or `ANTHROPIC_API_KEY` .
- **Request Timeout**: Configured with a 30-second `AbortSignal.timeout(30_000)` .
- **Deterministic Rule-Based Fallback**:

- If `CLAUDE_API_KEY` is not provided, or if Anthropic API returns a non-200 code or times out, the backend executes internal rule-based algorithms (`generateRuleInsights` , `generateRuleSummary` , `generateRuleSuggestions`).

- These inspect user history, compute rolling completion rates, detect missed habits, and generate structured suggestions without failure.

- Results are tagged with `source: 'rules'` and cached in `AiCache` for the configured cache duration.

Testing Setup

- **Framework**: Node.js native test runner (`node:test`) with `node:assert/strict` .
- **Test File**: `server/test/logic.test.mjs` (188 lines).
- **Coverage**: 22 unit tests covering:

- XP leveling curve calculations and monotonicity.

- Letter grade boundaries (S, A, B, C, D).

- Current streak and best streak backward evaluation (80% daily threshold).

- Schedule rule parsing (daily, weekdays, weekends, custom, once).

- Task progress and XP cap boundaries.

- Focus session XP formulas and pause penalties.

- **Execution Command**: `npm --prefix server test` or `npm run test:api` .

---

4. FRONTEND DETAILS

Next.js App Router Pages & Routes

1. `/` **(** `src/app/page.jsx` **)**: Root index route. Inspects auth status and routes authenticated users to `/quests` or renders the login view.

2. `/quests` **(** `src/app/quests/page.jsx` **,** `loading.jsx` **,** `error.jsx` **)**: Core quest management view. Renders `DayStrip` , `TaskList` , filter tabs, search toolbar, top KPI cards, and `CcompanionSidebar` .

3. `/focus` **(** `src/app/focus/page.jsx` **,** `loading.jsx` **,** `error.jsx` **)**: Focus Chamber view. Renders a radial Pomodoro progress ring, phase indicator, task selector, audio chimes toggle, and controls.

4. `/analytics` **(** `src/app/analytics/page.jsx` **,** `loading.jsx` **,** `error.jsx` **)**: SaaS performance dashboard. Contains lifetime KPI stats, 18-week heatmap, Recharts trend charts, sortable/filterable session history table with CSV export, and habit consistency rankings.

5. `/achievements` **(** `src/app/achievements/page.jsx` **,** `loading.jsx` **,** `error.jsx` **)**: Trophies view. Renders unlocked/locked badge tiles, bonus XP indicators, unlock timestamps, and milestone progress.

6. `/settings` **(** `src/app/settings/page.jsx` **,** `loading.jsx` **,** `error.jsx` **)**: User preferences view. Provides profile information, theme mode toggle, audio chime controls, timer interval configuration, JSON backup import/export, and progress reset.

7. `/login` **(** `src/app/login/page.jsx` **)**: Dedicated authentication view. Features a split-screen layout with an interactive dashboard mockup, value proposition cards, demo mode credentials pill, and theme toggle.

Shared / Reusable UI Components

- `PageHeader.jsx` : Standardized page title header with small-caps breadcrumb eyebrow, title, status badges, description, and right-aligned actions.

- `StatCard.jsx`: Standardized KPI card featuring uppercase tracking labels, large numbers, Lucide icons, and trend indicators.
- `EmptyState.jsx`: Dashed container placeholder with icon, message, and call-to-action button for empty data states.
- `Skeleton.jsx`: Modular skeleton loading components (`Skeleton`, `SkeletonCard`, `SkeletonKPI`, `SkeletonTable`).
- `ErrorState.jsx`: Standardized error card with retry button action.
- `ConfirmDialog.jsx`: Accessible confirmation modal for destructive operations.
- `QuestCard.jsx`: Interactive card for a single quest with completion checks, XP badges, timer/counter controls, and edit/delete actions.
- `Badge.jsx`: Achievement badge tile with lock status, emoji avatar, XP reward pill, and unlock timestamp.
- `ProgressRing.jsx`: Radial SVG progress circle for percentage representation.
- `Confetti.jsx`: Canvas-confetti milestone celebration trigger.
- `CommandPalette.jsx`: Quick navigation and action command palette (`cmdk`).
- `AppShell.jsx`: Shell wrapper providing responsive layout, drawer, sidebar, topbar, mobile nav, and route guarding.
- `Sidebar.jsx`: Persistent left navigation with grouped SaaS sections (`TRACK`, `REVIEW`, `ACCOUNT`), active indicators, and user card.
- `TopBar.jsx`: Sticky top navigation bar with streak, XP progress pill, and theme switcher.

Frontend-to-Backend Communication

- **API Client**: Implemented in `src/api/client.js` with base URL resolving from `NEXT_PUBLIC_API_URL` (defaults to `http://localhost:4000/api/v1`).
- **Token Storage**: JWT tokens are stored in `localStorage` under `dailyquest:auth` and mirrored in the `dailyquest_auth` cookie for routing synchronization.
- **Token Header**: Sent as `Authorization: Bearer <token>` on all requests when the user is not in guest mode.
- **Offline Sync & Storage (`src/storage.js`)**:

- `localStorage` under `dailyquest:v1` serves as the immediate read/write cache.

- State changes are debounced (700ms delay) and flushed to the backend via `PUT /api/v1/state`.

- Failed network requests are queued in `localStorage` under `dailyquest:pending` and synced when `window.addEventListener('online')` triggers.

- Background state refresh dispatches custom window event `'dailyquest:server-state'`.

State Management Approach

- **Architecture**: React Context API with modular custom providers:

- `AuthContext` (`useAuth`): Handles user identity, login, signup, guest sessions, logout, and client route guards.

- `QuestContext` (`useQuest`): Main centralized state reducer managing tasks, XP calculation, daily history entries, focus sessions, badges, settings, and task timers.

- `FocusContext` (`useFocus`): Focus room timer engine managing countdown intervals, active task linkage, and background ticks.

- `ModalContext` (`useModals`): Controls modal states (New Quest, Edit Quest, Finish Day Report, Confirm Dialog).

Styling Approach

- **Engine**: Tailwind CSS v4 using `@tailwindcss/postcss` and CSS custom properties in `src/app/globals.css`.
- **Dark Mode**: Configured via `@custom-variant dark (&:where(.dark, .dark *));`. Toggled by adding or removing the `.dark` class on `document.documentElement`.
- **Theme Palette**: Purple/indigo primary brand palette (`#6366f1`), obsidian dark background (`#090d16`), slate neutral tones, and accent highlights (amber, emerald, rose).
- **Design System Rules**: 8px spatial grid scale, `rounded-xl` / `rounded-2xl` card containers, `shadow-xs` card depths, and uppercase small-caps tracking headers.

---

5. FEATURES IMPLEMENTED SO FAR

Based on direct code inspection, the following features are fully implemented and verified operational:

- ✓ **Multi-Page App Router Structure**: Full Next.js routing split across `/quests`, `/focus`, `/analytics`, `/achievements`, `/settings`, and `/login`.
- ✓ **Authentication Flow**: Account signup and login with bcrypt hashing and JWT token issuance; Guest session mode with zero setup.
- ✓ **Session Sign-Out & Route Guard**: Functional sign-out from all three entry points (Sidebar, TopBar, Settings) with session clearing and route redirection.
- ✓ **Task CRUD**: Complete task creation, reading, editing, and deletion with confirmation dialogs.
- ✓ **Flexible Repeat Rules**: Task scheduling with `daily`, `weekdays`, `weekends`, `custom` (multi-day select), and `once` (calendar date).
- ✓ **Task Interaction Types**: Binary checkboxes (`yesno`), numeric target counters with custom units (`count`), and timed stopwatch/countdown tasks (`timed`).
- ✓ **Time of Day Scheduling**: Filtering and organization by `any`, `morning`, `afternoon`, and `evening`.
- ✓ **Gamified Progression Engine**: Proportional XP awards, level progression formulas, and level progress bar.
- ✓ **Streak Calculation**: Continuous streak tracking with 80% daily completion threshold.
- ✓ **Achievement Badges**: Rule-driven badge unlocking with reward notifications.
- ✓ **Focus Room & Pomodoro Engine**: 25m Pomodoro, custom intervals, stopwatch mode, audio chimes, and automatic focus session logging.

- ✓ **End-of-Day Review**: Day finalization with performance grading (S/A/B/C/D) and +50 XP perfect day bonuses.
- ✓ **Performance Analytics**: 18-week activity heatmap, Recharts trend charts (daily, weekly, monthly), and habit consistency rate rankings.
- ✓ **Structured Audit History Table**: Sortable and filterable activity table with search and direct CSV data export.
- ✓ **Data Backup & Restore**: Full client state backup download and restore via JSON.
- ✓ **Dark / Light Theme System**: Complete theme toggle with persistence in settings.
- ✓ **Offline Synchronization**: LocalStorage caching with background synchronization and pending write queues.
- ✓ **Backend API & Database Persistence**: Express.js REST API with automatic MongoDB in-memory provisioning.
- ✓ **Command Palette**: Keyboard-driven command bar (`cmdk`) for fast app navigation.

---

6. KNOWN GAPS OR RISKS

Factual list of items identified in the codebase that are unhandled, incomplete, or present technical risks:

1. Absence of Frontend Tests: The `src/` directory contains zero automated unit, integration, or end-to-end tests (no Jest, Vitest, or Playwright setup).

2. Missing Backend Rate Limiting: The Express server does not implement rate limiting on authentication routes (`/api/v1/auth/login`, `/signup`) or AI routes (`/api/v1/ai/*`), leaving them vulnerable to credential brute-forcing or API quota exhaustion.

3. Absence of Security Headers: Express security headers via `helmet` (CSP, HSTS, X-Content-Type-Options) are not configured.

4. No Password Recovery: The authentication system has no password reset, email recovery, or forgotten passcode flow; users who lose credentials cannot recover accounts.

5. No Email Verification: User signups do not collect or verify email addresses; uniqueness is enforced solely by username.

6. Loose Database Schema for Game State: In `server/src/models/State.js`, tasks, history, focus sessions, and badges are typed as `mongoose.Schema.Types.Mixed`, bypassing database-level type constraints in favor of client offline compatibility.

7. Potential Last-Write-Wins Race Condition: Full state updates use `PUT /api/v1/state` with entire document replacement; concurrent edits across two devices could lead to the last save overwriting the other device's changes.

8. Browser Autoplay Audio Policy: Focus room victory chimes rely on Web Audio API `AudioContext`, which can be blocked by browser autoplay restrictions if triggered before user interaction on that page.

9. Dual Build Systems Present: The codebase contains both Next.js App Router (`next.config.mjs`) and legacy Vite (`vite.config.js`, `index.html`) configurations, which may cause developer confusion if scripts are run interchangeably.

10. Client-Side Auth Hydration Flash: Initial page renders display an unauthenticated state for a few milliseconds while `localStorage` is read on client mount, before hydrating the user session.

---

7. DEPENDENCIES

Frontend (`package.json`)

Framework & Core

- `next` : `^14.2.15` — React App Router framework.
- `react` : `^18.3.1` — UI rendering library.
- `react-dom` : `^18.3.1` — React DOM bindings.

UI & Visualization

- `lucide-react` : `^1.47.0` — Icon set.
- `recharts` : `^2.15.4` — Data visualization and charting.
- `canvas-confetti` : `^1.9.4` — Particle celebration effects.
- `sonner` : `^2.0.8` — Toast notifications.
- `cmdk` : `^1.1.1` — Command palette.

Styling

- `tailwindcss` : `^4.1.11` — Utility-first CSS framework.
- `@tailwindcss/postcss` : `^4.3.3` — PostCSS plugin for Tailwind CSS v4.
- `@tailwindcss/vite` : `^4.1.11` — Vite plugin for Tailwind CSS v4.
- `postcss` : `^8.5.28` — CSS transformation pipeline.
- `autoprefixer` : `^10.6.1` — Vendor prefixing.

Dev Tools & Fallback Build

- `vite` : `^6.3.5` — Alternative fast build tool.
- `@vitejs/plugin-react` : `^4.6.0` — React plugin for Vite.
- `concurrently` : `^10.0.5` — Concurrently run frontend and backend processes.

Backend (`server/package.json`)

Framework & Core

- `express` : `^4.21.2` — REST API web server.
- `cors` : `^2.8.5` — Cross-Origin Resource Sharing.
- `dotenv` : `^16.4.5` — Environment variable loader.

Database

- `mongoose` : `^8.9.5` — MongoDB object modeling and database connection.

Auth & Security

- `bcrypt` : `^5.1.1` — Password hashing.
- `jsonwebtoken` : `^9.0.2` — JWT token generation and verification.

Validation

- `zod` : `^3.24.1` — Schema validation for HTTP requests.

Dev Tools & Testing

- `mongodb-memory-server` : `^10.1.4` — In-memory / embedded MongoDB server for zero-setup local development.
- `node:test` (Native Node.js built-in) — Unit testing engine.